

Membaca Arsitektur Next.js + NestJS

Dokumen ini menjelaskan seluruh struktur proyek geobi-v2 dari nol — apa fungsi tiap folder dan file, bagaimana Next.js (frontend) dan NestJS (backend) saling terhubung, serta panduan praktis kalau kamu ingin menambah atau mengubah fitur sendiri.



Daftar Isi

01	Gambaran Besar: Kenapa Ada Dua Aplikasi	konsep
02	Struktur Monorepo (Folder Paling Luar)	peta
03	NestJS — Bagian Backend	apps/server
04	Next.js — Bagian Frontend	apps/client
05	packages/shared — Kamus Bersama	apps/client + apps/server
06	Alur Nyata #1: Proses Login	end-to-end
07	Alur Nyata #2: Membuka Dashboard PBB-P2	end-to-end
08	Panduan Praktis: Kalau Aku Mau...	cara kerja
09	Lembar Referensi Cepat	cheat sheet

01 Gambaran Besar: Kenapa Ada Dua Aplikasi

Laravel itu satu aplikasi yang mengurus semuanya sekaligus: routing, tampilan HTML, dan query database jadi satu. Di stack baru ini, tanggung jawab itu dipisah jadi dua aplikasi terpisah yang saling bicara lewat HTTP — ini pola yang sangat umum di dunia JavaScript modern.

Next.js apps/client

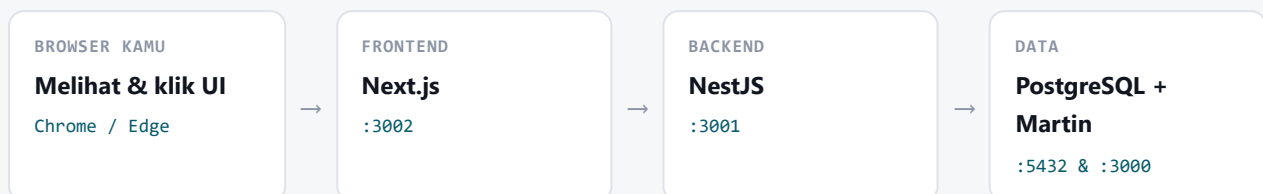
Next.js adalah **framework React**. Tugasnya cuma satu: menampilkan UI ke browser — halaman login, dashboard, peta, form profil, dan seterusnya. Next.js tidak pernah langsung menyentuh database. Jalan di `http://localhost:3002`.

NestJS apps/server

NestJS adalah **framework backend** (mirip semangat Laravel, tapi pakai TypeScript). Tugasnya menerima request HTTP, cek autentikasi, query ke PostgreSQL, dan mengembalikan JSON. Nest tidak pernah merender HTML. Jalan di `http://localhost:3001`.

Kenapa dipisah, bukan digabung?

Karena dua tanggung jawab itu beda sifatnya: Next.js dioptimalkan untuk menyusun UI dan dijalankan sebagian di browser (client-side), sementara Nest dioptimalkan untuk logika bisnis dan akses database yang harus tetap rahasia di server. Memisahkannya juga berarti kamu bisa men-deploy, men-scale, atau restart salah satunya tanpa mengganggu yang lain.



CATATAN PENTING

Martin adalah program terpisah (bukan bagian dari Next atau Nest) yang mengubah data peta di PostgreSQL/PostGIS menjadi "vector tiles" yang bisa langsung digambar MapLibre di peta. Next.js memanggil Martin **langsung dari browser** untuk ambil ubin peta — ini satu-satunya request yang tidak lewat Nest, karena data peta memang publik/tidak sensitif per-baris.

02 Struktur Monorepo (Folder Paling Luar)

"Monorepo" artinya satu folder Git berisi beberapa proyek terpisah yang saling terkait. Diatur oleh pnpm workspaces — ini yang membuat packages/shared bisa langsung di-import dari apps/client maupun apps/server tanpa publish ke npm.

```
geobi-v2/
├─ apps/
│   ├─ client/      ← Next.js, port 3002
│   └─ server/      ← NestJS, port 3001
├─ packages/
│   └─ shared/      ← kode & tipe dipakai bersama
├─ pnpm-workspace.yaml ← daftar mana saja yang termasuk workspace
├─ package.json      ← script bersama: pnpm dev, pnpm build...
└─ tsconfig.base.json ← setting TypeScript dasar yang diwarisi semua
```

FILE / FOLDER	FUNGSI
pnpm-workspace.yaml	Mendaftarkan apps/* dan packages/* sebagai satu workspace, supaya pnpm tahu mereka saling terkait dan bisa saling import.
package.json (root)	Cuma berisi script orkestrasi. pnpm dev di root = menjalankan dev di client dan server sekaligus, paralel.
tsconfig.base.json	Aturan TypeScript dasar (strict mode, target, dll) yang di-extend oleh tsconfig masing-masing app, supaya konsisten.

Cara menjalankan semuanya

Dari folder D:\laravelclone\geobi-v2, cukup satu perintah:

```
pnpm dev
```

Ini menyalakan Next.js dan NestJS bersamaan (lihat --parallel --filter ./apps/* di package.json). Martin (server peta) dijalankan terpisah lewat MARTIN\martin.exe karena bukan bagian dari kode Node kita.

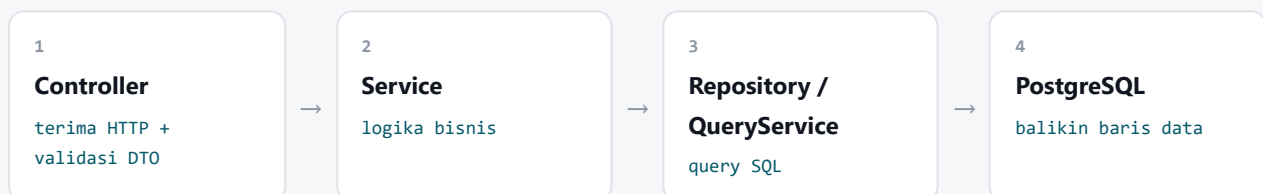
03 NestJS — Bagian Backend

Kalau kamu pernah pegang Laravel: **Controller** di Nest ≈ Controller di Laravel, **Service** ≈ kelas logic/Repository, dan **Module** mengelompokkan keduanya seperti sebuah "fitur" berdiri sendiri (mirip folder per-domain).

```
apps/server/src/
├─ main.ts                ← titik masuk paling awal, "nyalakan server"
├─ app.module.ts          ← module akar, mendaftarkan semua module lain
├─
├─ database/
│  ├─ pool.provider.ts     ← buka koneksi ke PostgreSQL (satu pool dipakai bersama)
│  ├─ query.util.ts       ← QueryService: helper query(sql, params) / one(sql, params)
│  └─ database.module.ts   ← ekspor koneksi itu supaya module lain bisa pakai
├─
├─ auth/                  ← login, register, refresh token, lupa password
│  ├─ auth.controller.ts   ← terima HTTP request (POST /auth/login, dst)
│  ├─ auth.service.ts      ← logika: cek password, bikin JWT, dll
│  ├─ auth.module.ts
│  ├─ dto/                 ← bentuk data yang divalidasi (LoginDto, RegisterDto)
│  ├─ guards/jwt-auth.guard.ts ← "satpam" — blokir request tanpa token valid
│  └─ strategies/jwt.strategy.ts ← cara membaca & verifikasi isi JWT
├─
├─ users/                 ← profil user: update nama/email/password, hapus akun
│  ├─ users.controller.ts
│  ├─ users.service.ts
│  └─ users.repository.ts  ← query SQL mentah ke tabel users
├─
├─ pbb/                   ← semua data & statistik PBB-P2 (inti aplikasi)
│  ├─ pbb.controller.ts    ← GET /pbb/stats, /pbb/search, /pbb/wilayah/*
│  ├─ pbb-stats.service.ts ← hitung agregat (total, realisasi, target, dst)
│  ├─ pbb-search.service.ts ← cari bidang tanah by alamat/nama/ID objek pajak
│  ├─ pbb-wilayah.service.ts ← daftar pilihan kelurahan/RW/RT (untuk dropdown filter)
│  ├─ filters.builder.ts   ← ubah filter dari UI jadi klausa SQL WHERE
│  └─ alamat-normalizer(.sql).ts ← "JL." → "jalan" dst, biar pencarian alamat toleran
├─
└─ common/filters/http-exception.filter.ts ← format error jadi JSON rapi
```

Siklus satu request di Nest

Setiap endpoint selalu melewati urutan yang sama:



Lalu hasilnya mengalir balik ke atas: Service mengubahnya jadi bentuk yang rapi, Controller membungkusnya jadi JSON, dan Nest mengirimkannya sebagai response HTTP.

Contoh nyata – apps/server/src/auth/auth.controller.ts

```
@Post('login')
async login(@Body() dto: LoginDto) {
  const user = await this.authService.validateUser(dto.email, dto.password);
  const tokens = await this.authService.issueTokens(user);
  return { ...tokens, user };
}
```

`@Post('login')` artinya route ini menjawab POST `/auth/login`. `@Body()` `dto: LoginDto` LoginDto otomatis mem-validasi body request sesuai bentuk LoginDto (kalau email kosong, Nest langsung menolak dengan error 400 sebelum kode kamu sempat jalan). Controller ini sendiri tidak tahu *bagaimana* password dicek — itu didelegasikan ke `auth.service.ts`.

KENAPA DIPISAH CONTROLLER VS SERVICE?

Controller cuma "penerima telepon" — ngangkat telepon, cek siapa yang nelpn (validasi), lalu oper ke orang yang tepat (Service). Service adalah orang yang benar-benar tahu cara kerjanya. Pemisahan ini memudahkan kamu *menemukan* logika: kalau soal query database, cari di Service/Repository. Kalau soal "endpoint apa saja yang ada", cari di Controller.

Guard & Strategy – cara Nest mengenali "siapa yang login"

`@UseGuards(JwtAuthGuard)` ditaruh di atas endpoint yang butuh login (misalnya GET `/pbb/stats`). Guard ini membaca header `Authorization: Bearer <token>`, lalu `jwt.strategy.ts` yang memverifikasi tanda tangan token itu dan mengambil `userId` dari dalamnya. Kalau token tidak valid/kadaluarsa, request ditolak sebelum masuk ke Controller sama sekali.

04 Next.js — Bagian Frontend

Next.js yang kita pakai menggunakan **App Router** (folder `src/app/`).

Aturan dasarnya sederhana: **satu folder = satu segmen URL**, dan file `page.tsx` di dalamnya adalah halaman yang ditampilkan untuk URL itu.

```
apps/client/src/
├─ middleware.ts           ← "satpam" tingkat aplikasi, jalan SEBELUM halaman dirender
├─ app/
│  ├─ layout.tsx           ← bungkus SEMUA halaman (html, body, provider)
│  ├─ globals.css          ← Tailwind + CSS peta, dimuat sekali di Layout
│  └─ (public)/            ← kurung = tidak muncul di URL, cuma pengelompokan
│     ├─ page.tsx           ← "/" (beranda)
│     ├─ tentang/page.tsx  ← "/tentang"
│     ├─ struktur/page.tsx ← "/struktur"
│     └─ katalog/page.tsx  ← "/katalog"
│
│  ├─ (auth)/              ← halaman sebelum login
│     ├─ login/page.tsx
│     ├─ register/page.tsx
│     └─ forgot-password/, reset-password/[token]/
│
│  ├─ (protected)/         ← WAJIB login (dijaga middleware.ts)
│     ├─ profile/page.tsx
│     └─ katalog/
│        ├─ dashboard-pbb/page.tsx ← "/katalog/dashboard-pbb" (Vol.2)
│        ├─ dashboard-pbb/v1/page.tsx ← Vol.1
│        ├─ dashboard-pbb/v3/page.tsx ← Vol.3
│        └─ pbb-p2/page.tsx        ← peta interaktif lama
│
│  └─ api/                 ← "Route Handlers" — backend KECIL di sisi Next
│     ├─ auth/login/route.ts, logout/, refresh/, me/, ...
│     ├─ pbb/stats/route.ts, search/, wilayah/...
│     └─ users/me/route.ts, me/password/route.ts
│
├─ features/              ← satu folder = satu fitur besar, isinya lengkap sendiri
│  ├─ dashboard-pbb/       ← Dashboard Vol. 2 (independen, lihat bab 8)
│  ├─ dashboard-pbb-v3/    ← Dashboard Vol. 3 (independen, salinan terpisah)
│  ├─ dashboard-v1/        ← Dashboard Vol. 1 (struktur beda sendiri)
│  └─ pbb-p2/              ← Peta interaktif lama
│
├─ components/            ← UI dipakai di BANYAK halaman
│  └─ Navbar.tsx, SiteFooter.tsx, DarkModeToggle.tsx, QueryProvider.tsx
│
└─ lib/                   ← fungsi bantuan, tidak ada tampilan
   ├─ fetchNest.ts         ← helper: panggil Nest dari Route Handler
   ├─ cookies.ts           ← nama cookie & alamat Nest
   └─ useAuthUser.ts       ← React hook: siapa user yang sedang login
```

Empat lapis yang wajib kamu pahami

1) middleware.ts – satpam paling depan

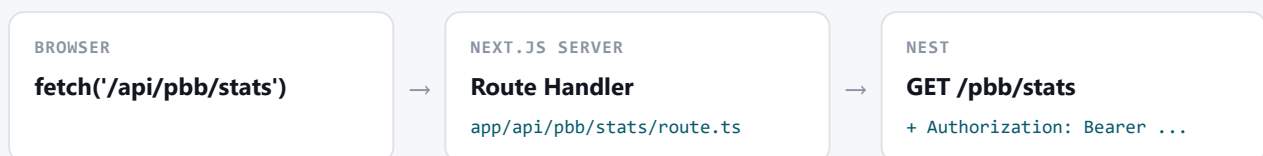
File ini jalan **di server, sebelum halaman apa pun dirender**, untuk setiap request yang cocok dengan pola di matcher (saat ini: `/katalog/dashboard-pbb/*`, `/katalog/pbb-p2/*`, `/profile/*`). Isinya: cek cookie login ada atau tidak → kalau tidak ada, coba refresh token diam-diam → kalau tetap gagal, redirect ke `/login`. Ini sebabnya kamu tidak akan pernah "kelihatan sekilas" halaman dashboard sebelum dilempar ke login — keputusan sudah dibuat sebelum HTML dikirim.

2) Route Groups – (public), (auth), (protected)

Folder berkurang **tidak muncul di URL** sama sekali — fungsinya cuma mengelompokkan halaman yang berbagi `layout.tsx` yang sama. Misalnya `app/(public)/layout.tsx` memasang tema gelap + navbar publik untuk semua halaman di dalam (public), sedangkan `(protected)/layout.tsx` tidak menambah apa-apa karena setiap fitur di dalamnya (dashboard, profil) sudah membawa layoutnya sendiri-sendiri.

3) app/api/* – kenapa ada "backend kecil" di dalam frontend?

Ini bagian yang paling sering membingungkan orang baru. Browser **tidak pernah** memanggil NestJS secara langsung untuk data yang butuh login. Alurnya selalu:



Alasannya keamanan: token login (JWT) disimpan sebagai cookie `httpOnly`, yang artinya **JavaScript di browser sama sekali tidak bisa membacanya** (kebal dari serangan XSS). Hanya kode yang jalan di server Next.js yang boleh membaca cookie itu dan menempelkannya sebagai header `Authorization` saat memanggil Nest. Route Handler inilah "penjaga" yang melakukan itu, lewat helper `fetchNest()`.

apps/client/src/app/api/pbb/stats/route.ts

Contoh Route Handler paling sederhana — hanya meneruskan query string apa adanya ke Nest, lalu mengembalikan hasilnya:

```
export async function GET(request: Request) {
  const { search } = new URL(request.url);
  const res = await fetchNest(`/pbb/stats${search}`);
  const data = await res.json();
  return NextResponse.json(data, { status: res.status });
}
```

4) features/ vs components/ vs lib/ – ke mana file baru harus masuk?

FOLDER	ISINYA APA	ATURAN SEDERHANA
features/<nama>/	Semua yang berhubungan dengan satu fitur besar: komponen, hook, CSS, state (Zustand), fungsi helper khusus fitur itu.	Kalau file itu <i>hanya</i> dipakai oleh satu dashboard, taruh di sini.
components/	Komponen UI yang dipakai lintas fitur/halaman.	Kalau dipakai di ≥ 2 tempat berbeda (Navbar dipakai di semua halaman), naikkan ke sini.

lib/ Fungsi murni, tidak ada JSX/tampilan.

Kalau tidak me-render apa pun (fetch helper, format tanggal, cek cookie), taruh di sini.

05 packages/shared — Kamus Bersama

Beberapa hal harus **persis sama** di Next.js maupun NestJS — misalnya daftar status pembayaran, atau singkatan alamat ("Jl." → "jalan"). Kalau ditulis dua kali di dua tempat, cepat atau lambat keduanya akan menyimpang (ini yang dulu bikin dashboard v1/v2/v3 di Laravel jadi beda-beda tanpa alasan jelas). `packages/shared` menyimpan definisi itu satu kali saja.

```
packages/shared/src/
├─ constants/
│   ├─ status-pem.ts      ← string status: "SUDAH BAYAR", "BELUM BAYAR / BELUM LUNAS", dst
│   └─ alamat-abbr.ts    ← Map singkatan alamat, dipakai JS (client) & SQL (server)
├─ dto/
│   ├─ stats.ts          ← bentuk data statistik (PbbAggregateRow, dst)
│   ├─ search.ts
│   └─ wilayah.ts
└─ index.ts              ← "pintu keluar" – semua export dikumpulkan di sini
```

Dari `apps/server` maupun `apps/client`, package ini dipanggil dengan nama `@geobi/shared`, contoh:

```
import { PbbAggregateRow } from '@geobi/shared';
```

YANG PERLU DIINGAT

Package ini di-*compile* dulu (TypeScript → JavaScript biasa) sebelum dipakai server, karena Node tidak bisa langsung menjalankan file `.ts` mentah dengan cara yang dipakai di sini. Kalau kamu mengedit sesuatu di `packages/shared` dan perubahannya tidak "kelihatan" di server, coba build ulang package itu.

06 Alur Nyata #1: Proses Login

Supaya konsep di atas terasa konkret, ini urutan file yang benar-benar dilewati satu klik tombol "Masuk", dari ujung ke ujung.

1 User isi form & klik "Masuk"

`app/(auth)/login/page.tsx`

Form React memanggil `fetch('/api/auth/login', { method: 'POST', body: {...} })` — ini alamat di Next.js sendiri, BUKAN ke Nest.

2 Route Handler menerima, lalu meneruskan ke Nest

`app/api/auth/login/route.ts`

Memanggil POST `http://localhost:3001/auth/login` dengan email/password apa adanya.

3 Nest menerima & validasi bentuk data

`apps/server/src/auth/auth.controller.ts` → `@Post('login')`

DTO `LoginDto` memastikan email & password terisi sebelum lanjut.

4 Service mencocokkan password & membuat token

`apps/server/src/auth/auth.service.ts`

`validateUser()` query tabel `users`, cocokkan password pakai `bcryptjs.compare()` (kompatibel dengan hash `bcrypt` lama dari Laravel — akun lama tidak perlu reset password). Kalau cocok, `issueTokens()` membuat access token (berumur pendek) & refresh token (berumur panjang).

5 Nest membalas JSON berisi token + data user

Response ini kembali ke Route Handler di langkah 2, bukan langsung ke browser.

6 Route Handler menyimpan token sebagai cookie `httpOnly`

`app/api/auth/login/route.ts`

`cookieStore.set('geobi_access', token, { httpOnly: true, ... })`. Ke browser, yang dikirim balik cuma `{ user }` — token mentah tidak pernah sampai ke JavaScript sisi klien.

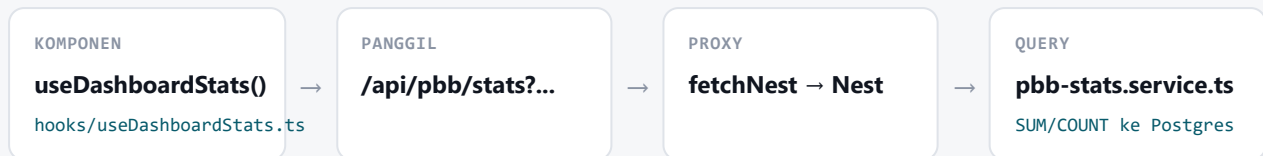
7 React redirect ke dashboard

Kunjungan berikutnya ke halaman terproteksi akan lolos `middleware.ts` karena cookie access token sudah ada.

07 Alur Nyata #2: Membuka Dashboard PBB-P2

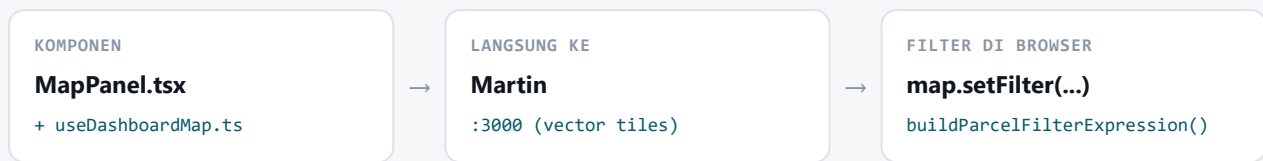
Contoh kedua ini menunjukkan bagaimana data statistik (gauge, KPI) dan data peta datang dari **dua jalur yang berbeda** — ini penting dipahami karena keduanya harus tetap konsisten satu sama lain (lihat juga catatan soal filter kategori di bab 8).

Jalur A — angka di sidebar (gauge, KPI, distribusi)



Filter yang sedang aktif (kelurahan, tahun, kategori, dst) disimpan di `store/filters.store.ts` (Zustand). Setiap filter berubah, hook ini membangun ulang query string lewat `lib/build-stats-query.ts` dan fetch ulang — hasilnya dipakai `GaugeCard.tsx`, `KpiGrid.tsx`, dan `DonutChart.tsx`.

Jalur B — warna bidang tanah di peta



Peta **tidak** lewat Nest sama sekali. MapLibre mengambil semua ubin dari Martin, lalu filter (kelurahan/status/kategori) diterapkan sebagai ekspresi MapLibre langsung di browser lewat `map.setFilter()`. Ini kenapa cepat — tidak perlu bolak-balik ke server tiap geser peta.

KENAPA DUA JALUR INI HARUS DIJAGA KONSISTEN

Karena sumber datanya beda (Jalur A: agregat SQL di Nest, Jalur B: ekspresi filter di MapLibre), logika "apa yang cocok dengan filter X" harus ditulis dua kali dengan hasil yang identik — satu di `filters.builder.ts` (server, SQL), satu di `hooks/useDashboardMap.ts` fungsi `buildParcelFilterExpression()` (client, MapLibre). Ini persis bug yang pernah kita perbaiki: saat semua checkbox kategori di-uncheck, jalur peta sudah benar menampilkan 0 bidang, tapi jalur statistik sempat lupa ikut menampilkan 0.

08 Panduan Praktis: Kalau Aku Mau...

Skenario paling umum yang akan kamu temui, dan file mana saja yang perlu disentuh.

...menambah halaman statis baru (misalnya "/faq")

1. Buat folder `app/(public)/faq/page.tsx` — otomatis jadi route `/faq`, tanpa daftar di mana pun.
2. Kalau perlu link di menu, tambahkan di `components/Navbar.tsx`.

...menambah endpoint API baru (misalnya statistik baru)

1 Tulis logikanya di Nest

`apps/server/src/pbb/pbb-stats.service.ts`

2 Buka endpoint di Controller

`apps/server/src/pbb/pbb.controller.ts` — tambah `@Get('stats/baru')`

3 Buat "jembatan" di Next

`app/api/pbb/stats-baru/route.ts` — proxy pakai `fetchNest()` seperti contoh di bab 4

4 Panggil dari komponen

`fetch('/api/pbb/stats-baru')` dari React — biasanya dibungkus TanStack Query lewat hook baru di `features/<nama>/hooks/`

...mengubah tampilan Dashboard Vol. 2 tanpa menyentuh Vol. 3

Sejak dipisah, keduanya adalah dua salinan berdiri sendiri — edit bebas tanpa saling pengaruh:

MAU UBAH	VOL. 2	VOL. 3
Komponen & hook	<code>features/dashboard-pbb/</code>	<code>features/dashboard-pbb-v3/</code>
Komponen utama	<code>DashboardPbb.tsx</code>	<code>DashboardPbbV3.tsx</code>
Pengaturan tampilan (gauge, tour, dll)	<code>config.ts</code> → export CONFIG	<code>config.ts</code> → export CONFIG (nilai beda)
Style CSS	<code>dashboard.css</code>	<code>dashboard.css</code>
Route	<code>/katalog/dashboard-pbb</code>	<code>/katalog/dashboard-pbb/v3</code>

Vol. 1 (`features/dashboard-v1/`) sudah dari awal berdiri sendiri karena sidebar & pencariannya memang beda struktur, bukan sekadar beda pengaturan.

...menambah kolom baru di query database

1. Tambahkan kolom itu ke SELECT di `apps/server/src/pbb/pbb-stats.service.ts` (atau service terkait).
2. Kalau bentuk datanya perlu diketahui TypeScript, tambahkan field-nya di `packages/shared/src/dto/stats.ts` (interface `PbbAggregateRow`) supaya client & server sama-sama tahu tipe barunya.

3. Pakai field baru itu di komponen React yang relevan, misalnya `KpiGrid.tsx`.

...menambah pilihan filter baru di panel

1. Tambah state-nya di `store/filters.store.ts` (Zustand) milik fitur terkait.
2. Tambah UI checkbox/dropdown-nya di `components/MapPanel/panels/`.
3. Pastikan filter itu dibaca di **dua tempat** (lihat bab 7, Jalur A & B): `lib/build-stats-query.ts` (untuk sidebar) dan `hooks/useDashboardMap.ts` fungsi `buildParcelFilterExpression()` (untuk peta) — dan juga di server, `apps/server/src/pbb/filters.builder.ts`.

09 Lembar Referensi Cepat

Port yang dipakai

SERVIS	PORT	DIJALANKAN LEWAT
Next.js (client)	3002	pnpm dev
NestJS (server)	3001	pnpm dev
Martin (tile server peta)	3000	MARTIN\martin.exe
PostgreSQL	5432	Windows service (native, selalu nyala)

Istilah cepat

ISTILAH	ARTINYA DI PROYEK INI
page.tsx	Isi/tampilan sebuah halaman untuk route sesuai posisi foldernya.
layout.tsx	Bungkus yang dipakai bersama semua halaman di dalam foldernya.
route.ts (di dalam app/api)	Route Handler — endpoint backend kecil di Next, biasanya proxy ke Nest.
Controller (Nest)	Penerima HTTP request, tidak menyimpan logika berat.
Service (Nest)	Logika bisnis & akses data.
Module (Nest)	Kelompok Controller+Service yang berhubungan (auth, users, pbb).
DTO	Bentuk data yang divalidasi otomatis oleh Nest sebelum masuk ke Controller.
Guard	Pemeriksa akses sebelum Controller dijalankan (mis. wajib login).
httpOnly cookie	Cookie yang tidak bisa dibaca JavaScript di browser — tempat token JWT disimpan.
Zustand store	Tempat menyimpan state UI (filter aktif dll) yang dipakai banyak komponen sekaligus.
TanStack Query	Pengatur fetch data dari server + cache-nya di React.